Do Efficient Generative Sampling Choices Transfer Across GPUs?

Training-Free Acceleration of Diffusion and Flow-Matching Transformers

Iason-Daniil Zakynthinos

***Abstract*—Do quality–efficiency conclusions about diffusion and flow-matching sampling remain valid across GPU platforms when training-free accelerations are introduced? This project tests whether the preferred sampling configuration changes at fixed image quality, and measures the cost of transferring choices between devices. Using pretrained DiT and SiT transformers, I compare plain sampling, feature caching, and INT8/FP8 inference on three GPU platforms. The primary outcome is the transfer penalty: the extra latency incurred by transferring a source-device choice instead of selecting on the target. Comparisons separate kernel availability from performance on shared configurations. Execution-cost models and feature-reuse diagnostics are used to explain the changes.**

I. Research Question and Hypotheses

Do quality–efficiency conclusions about diffusion and flow-matching sampling remain valid across GPU platforms when training-free accelerations are introduced?

H1 (transfer baseline). Within each batch size, a device-specific multiplicative latency factor adequately describes plain sampling across platforms. This separable model predicts stable choices; configuration-by-device interactions are the competing explanation. The test concerns practical selection penalties, not merely unequal speedups.

H2 (cost explanation). Caching and low-precision execution introduce interactions that cannot be explained by NFE or operation count alone. A parsimonious model incorporating effective arithmetic throughput, memory traffic, and fixed execution overhead should better predict held-out latency and transferred choices. Interactions may exist without changing the preferred configuration; the transfer penalty measures whether they matter.

H3 (quality mechanism; exploratory). Relative feature-reuse error should predict caching-induced quality degradation better than step count alone, including when evaluated on the other model. No directional claim that flow matching tolerates caching better is assumed: straighter interpolation paths do not guarantee slowly changing hidden features.

II. Motivation and Related Work

Diffusion and flow-matching models generate samples through repeated network evaluations [1], [2]. DiT and SiT provide comparable transformer backbones while differing in their generative formulation [3], [4]. Numerical solvers reduce the number of evaluations [5], [6]; feature caching reduces computation within an evaluation [7], and post-training quantization changes its arithmetic cost [8]. These interventions therefore alter different components of inference cost.

Reported speedups are usually measured on one GPU and can be fragile: FORA gave no speedup ($0.99\times$) in a recent batch-1 comparison on an RTX 5090 [9]. Quality–cost frontiers and caching schedules are already studied [10], operation counts are known to be imperfect efficiency proxies [11], and inference energy is benchmarked across models and GPUs [12]. Hardware-dependent speedup alone is therefore not the proposed contribution. The open question is whether platform differences change quality-constrained decisions, by how much, and whether inexpensive measurements can predict those changes. DiT and SiT are treated as two related workloads, not as a causal test of their training objectives.

III. Methodology

Models and factors. Public ImageNet-256 DiT-XL/2 and SiT-XL/2 checkpoints; no retraining. Samplers are DDIM and a specified DPM-Solver variant for DiT, and Euler and Heun ODE for SiT. Each uses 8, 16, or 32 solver steps and one of four execution modes: FP16 baseline, FORA-style caching [7], INT8 W8A8, or FP8 W8A8. These are 48 configurations before availability exclusions. Batch sizes 1 and 16 are separate timing conditions. Actual NFE, including solver substages, guidance execution, and cached calls, is logged independently of nominal steps.

Hardware and implementation. An RTX 5080, a Colab T4, and a GH200 are three tested platforms, not samples of a GPU generation. End-to-end smoke tests establish supported kernels before the sweep. The manifest fixes checkpoint, timestep grid, solver order, guidance, labels, seeds, compilation policy, and VAE. Quantization specifies calibration data, scale granularity, excluded layers, accumulation precision, and the actual low-precision kernels. Calibration and evaluation seeds are disjoint; fake quantization is not counted as accelerated execution. PyTorch and torchao versions are pinned where possible, with driver, CUDA, host, and remaining stack differences recorded.

Caching policy and probes. Cached attention/MLP outputs, refresh interval, initial refresh, and solver-substage behavior are fixed before evaluation. On a small independent probe set, reused features are compared with full recomputation at the same state and time, logging relative L2 error, magnitude change, and timestep/layer summaries. Probes run outside timed workloads, and the diagnostics are related to the FID change against the corresponding uncached configuration.

Quality. Generate 2,000 images per configuration for screening, using clean-fid with fixed ImageNet reference

statistics [13] and precision/recall [14]. Before confirmation, freeze three FID ceilings spanning the useful pilot range and the selection rules. Retain configurations near the frontier, including those whose apparent dominance is unresolved. Confirm with three independent 10,000-image runs per retained configuration; report per-run scores and uncertainty rather than pooling unequal sample sizes. Equal counts do not eliminate FID bias [15].

Most quality evaluation uses the RTX 5080. Before reusing these scores elsewhere, regenerate a prespecified subset covering both models and every shared execution mode, including near-threshold cases. If equivalence within a declared tolerance is unresolved, evaluate the affected target-device candidates directly or exclude those comparisons from transfer claims.

Timing and energy. After warm-up and compilation, randomize configuration order and repeat measurements in multiple sessions. Report median and interquartile range for synchronized end-to-end generation, with denoising and VAE decoding also timed separately. Distinguish batch latency from throughput and amortized time per image. Record peak memory, power limits, and clocks. Energy is secondary: measure repeated workloads over long windows using supported NVML counters, state the power domain and idle treatment, and omit incomparable measurements.

IV. Analysis, Feasibility, and Expected Contribution

Primary outcome. For each ordered device pair, batch size, and FID ceiling, restrict selection to configurations executable on both platforms and meeting the quality requirement. Choose the fastest on the source and evaluate its target latency. Transfer penalty is target latency divided by the fastest eligible target latency, minus one. Define near-optimal as a penalty of at most 5%. Unsupported methods and empty feasible sets are reported separately. Propagate timing and quality uncertainty; do not force a winner when eligibility or dominance is unresolved.

H1/H2. Compare the common-scaling baseline with configuration-by-device interactions. Fit NFE-only and operation-count baselines, then a small hardware-aware cost model using compute, memory, and overhead terms. Effective rates come from independent representative kernel probes; memory traffic is estimated with stated assumptions. Validate by holding out configuration groups and, exploratorily, one device at a time, without fitting to its end-to-end timings. Report latency error and decision penalty. Pareto frontiers and latency-only Kendall tau-b on shared configurations are secondary summaries.

H3. Test whether feature-reuse diagnostics predict the quality change beyond model, solver, and step count. Evaluate cross-model prediction separately from within-model fit, and report negative results.

Feasibility. Nothing is trained. At DiT-XL/2 cost, the 2,000-image pilot over all 48 configurations needs roughly 6 GPU-hours on the RTX 5080 and the confirmation stage about a day; the exact budget is set from measured pilot throughput and logged. Work on the T4 and GH200 is limited to timing sweeps, kernel probes, and the regenerated quality subset. If time is short, cut in this order: held-out-device validation, FP8, batch 16 on the GH200, then the 32-step level for accelerated configurations. The transfer-penalty analysis on shared configurations is never cut.

Expected contribution. Evidence on whether the preferred model–sampler–acceleration configuration changes across platforms at fixed image quality; transfer penalties with uncertainty; an execution-cost explanation of the changes; and an open harness and manifest. Stable choices with tightly bounded penalties are also informative. Claims are limited to the tested platforms and configuration space. Serving, training, and exhaustive solver or cache-policy optimization are out of scope.

V. Proposed Schedule

Weeks 1–2: checkpoints, environments, complete kernel smoke tests, and manifest. *3–4:* solver/caching validation, quantization calibration, and timing protocol. *5–7:* quality pilot, feature probes, frozen thresholds and shortlist. *8–9:* cross-platform timing, kernel probes, and quality checks. *10–11:* independent quality confirmation and targeted target-device evaluation. *12–13:* held-out analysis, uncertainty, plots, and report.

References

[1] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. NeurIPS*, 2020.

[2] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *Proc. ICLR*, 2023.

[3] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. ICCV*, 2023.

[4] N. Ma, M. Goldstein, M. S. Albergo, N. M. Boffi, E. Vanden-Eijnden, and S. Xie, "SiT: Exploring flow and diffusion-based generative models with scalable interpolant transformers," in *Proc. ECCV*, 2024.

[5] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *Proc. ICLR*, 2021.

[6] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, "DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps," in *Proc. NeurIPS*, 2022.

[7] P. Selvaraju, T. Ding, T. Chen, I. Zharkov, and L. Liang, "FORA: Fast-forward caching in diffusion transformer acceleration," arXiv:2407.01425, 2024.

[8] J. Wu, H. Wang, Y. Shang, M. Shah, and Y. Yan, "PTQ4DiT: Post-training quantization for diffusion transformers," in *Proc. NeurIPS*, 2024.

[9] X. Yin, C. Xu, H. Zhou, B. Wei, and W. Cai, "AccelAes: Accelerating diffusion transformers for training-free aesthetic-enhanced image generation," in *Proc. ECCV*, 2026.

[10] A. Aggarwal, A. Shrivastava, and M. Gwilliam, "Evolutionary caching to accelerate your off-the-shelf diffusion model," in *Proc. ICLR*, 2026.

[11] M. Dehghani, A. Arnab, L. Beyer, A. Vaswani, and Y. Tay, "The efficiency misnomer," in *Proc. ICLR*, 2022.

[12] J.-W. Chung, J. Liu, J. J. Ma, R. Wu, O. J. Kweon, Y. Xia, Z. Wu, and M. Chowdhury, "The ML.ENERGY benchmark: Toward automated inference energy measurement and optimization," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2025.

[13] G. Parmar, R. Zhang, and J.-Y. Zhu, "On aliased resizing and surprising subtleties in GAN evaluation," in *Proc. CVPR*, 2022.

[14] T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, and T. Aila, "Improved precision and recall metric for assessing generative models," in *Proc. NeurIPS*, 2019.

[15] M. J. Chong and D. Forsyth, "Effectively unbiased FID and Inception Score and where to find them," in *Proc. CVPR*, 2020.